

Lab 9: Privacy Preserving in Federated Learning

In this lab, we will explore privacy preserving in the application of Federated Learning framework.

Federated Learning is a learning framework for multiple clients' Deep Learning models with their on-device data to which you cannot have access. Instead of collecting all clients' data into one machine and training a model, we bring the model to the data, train it locally, and merely upload "model updates" to a central server.

In this lab, we will achieve the following objectives.

Objectives:

1. Federated Learning: Understand the Federated Learning framework, including the role of the centralized server and clients, and the process of federated learning.
2. Tensor: Review "tensor" and its basic operations.
3. PySyft: What is PySyft; How to install PySyft; How to use PySyft; How to create virtual clients; How to aggregate clients' inputs.

1. Pre-work: Installing PySyft

In order to simulate a federated learning framework, we need to use Deep Learning techniques on remote machines. Specifically, we will use PySyft, an extension of PyTorch, to simulate multiple clients' deep learning process and the server's aggregation process.

1.1. Check Installation / Install PySyft

Nomally, we have installed PySyft for you. Check if PySyft is successfully installed in the terminal with the following command:

```
$pip3 list
```

If the PySyft is successfully installed, you can see its name and version in the list, e.g.

```
pysyft      0.0.1
```

If PySyft is not successfully installed on your computer, then install the dependencies by putting the following commands in your terminal:

```
$pip3 install pysyft
```

1.2. Testing task: Build your first tensor

A tensor is a generic multiple-dimensional array, very similar to numpy arrays, or matrix. Tensors are used for arbitrary numeric computation. Now, let's build your first tensor. Create a new python file in Jupyter notebook, and type the following code:

In [1]:

```
1 import torch
2
3 x = torch.tensor([1,1,1,1,1])
4
5 y = torch.tensor([[1,1,1,1,1], [1,1,1,1,1]])
6
7 print("tensor x is:", x)
8 print("tensor y is:", y)
```

```
tensor x is: tensor([1, 1, 1, 1, 1])
tensor y is: tensor([[1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1]])
```

In [2]:

```
1 import syft as sy
2
3 hook = sy.TorchHook(torch)
4
5 torch.tensor([1,2,3,4,5])
6
```

Out[2]:

```
tensor([1, 2, 3, 4, 5])
```

A tensor has a shape, to describe the number of dimensions and the length of each demension of this tensor. We can get the tensor's shape by:

In [3]:

```
1 print("tensor x's shape is:", x.shape)
2 print("tensor y's shape is:", y.shape)
```

```
tensor x's shape is: torch.Size([5])
tensor y's shape is: torch.Size([2, 5])
```

Tensors with the same shape can Participate in basic operations, e.g.

In [4]:

```
1 z = x + x
2
3 a = x * 2
4
5 print("New tensor z is:", z)
6 print("New tensor a is:", a)
7 print(a==z)
```

```
New tensor z is: tensor([2, 2, 2, 2, 2])
New tensor a is: tensor([2, 2, 2, 2, 2])
tensor([True, True, True, True, True])
```

2. Task-one: Create visual clients in PySyft

In this task, we will explore how to create visual clients for the federated learning. Federated learning allows

multiple clients to individually train deep learning model in parallel.

2.1. Create the first visual client for federated learning

We use PySyft to create our first client, "Alice".

In [5]:

```
1 alice = sy.VirtualWorker(hook, id = "alice")
2 alice._objects
```

Out[5]:

```
{}
```

In [6]:

```
1 alice_tensor = torch.tensor([1,1,1,1,1])
2 alice_tensor = alice_tensor.send(alice)
3 alice._objects
```

Out[6]:

```
{49557515688: tensor([1, 1, 1, 1, 1])}
```

Now, we can check Alice's information, such as location, id, owner with the following code:

In [7]:

```
1 print(alice_tensor.location)
2 print(alice_tensor.id_at_location)
3 print(alice_tensor.id)
4 print(alice_tensor.owner)
5 print(hook.local_worker)
```

```
<VirtualWorker id:alice #objects:1>
49557515688
71135165043
<VirtualWorker id:me #objects:0>
<VirtualWorker id:me #objects:0>
```

In [8]:

```
1 alice_tensor
```

Out[8]:

```
(Wrapper)>[PointerTensor | me:71135165043 -> alice:49557515688]
```

In [9]:

```
1 alice_tensor = alice_tensor.get()
2 alice_tensor
```

Out[9]:

```
tensor([1, 1, 1, 1, 1])
```

In [10]:

```
1 alice._objects
```

Out[10]:

```
{}
```

2.2. A single client's arithmetic

This section we will explore a client's learning process, e.g. basic operations, backward and gradients.

In [11]:

```
1 x = torch.tensor([1,1,1,1,1]).send(alice)
2 y = torch.tensor([1,-1,1,-1,1]).send(alice)
3
4 print(x)
5 print(y)
```

```
(Wrapper)>[PointerTensor | me:4127876707 -> alice:6747674158]
```

```
(Wrapper)>[PointerTensor | me:81506615267 -> alice:50731816953]
```

In [12]:

```
1 z = x + y
2 print(z)
```

```
(Wrapper)>[PointerTensor | me:32774676298 -> alice:19787982111]
```

In [13]:

```
1 z = z.get()
2 print(z)
```

```
tensor([2, 0, 2, 0, 2])
```

In [14]:

```
1 z = torch.add(x, y)
2 print(z)
```

```
(Wrapper)>[PointerTensor | me:92711507828 -> alice:1240157180]
```

In [15]:

```
1 z = z.get()
2 print(z)
```

```
tensor([2, 0, 2, 0, 2])
```

In [16]:

```
1 x = torch.tensor([1.,1,1,1,1], requires_grad = True).send(alice)
2 y = torch.tensor([1.,-1,1,-1,1], requires_grad = True).send(alice)
3 z = (x + y).sum()
```

In [17]:

```
1 x = x.get()
2 print(x)
3 print(x.grad)
```

```
tensor([1., 1., 1., 1., 1.], requires_grad=True)
None
```

Practice: aggregation of two clients' tensors

In this practice, we need you to create two clients, one is "Alex", one is "Bob". Alex has a tensor (1,2,3,4,5), and Bob has a tensor (1,1,1,1,1). We have the following tasks:

1. Create two clients "Alex" and "Bob", and allocate tensors for them.
2. Sum two clients' tensors and get a aggregated tensor.
3. Reallocate the aggregated tensor to two clients.

3. Task-two: An easy demo of Federated Learning

This section is an easy demo of federated learning. We train an easy linear model in a centralised way. In this demo, we have a dataset with data $\begin{bmatrix} 0,0 \\ 0,1 \\ 1,0 \\ 1,1 \end{bmatrix}$, $\begin{bmatrix} 0.1,0.1 \\ 0.1,1.1 \\ 1.1,0.1 \\ 1.1,1.1 \end{bmatrix}$, and its corresponding labels $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$.

In [18]:

```
1 import torch
2
3 # Training dataset
4 data = torch.tensor([[0,0],[0,1],[1,0],[1,1], [0.1,0.1],[0.1,1.1],[1.1,0.1],[1.1,1.1]])
5 target = torch.tensor([0,0,1,1, 0,0,1,1], requires_grad = True)
6
7 # Training model for the centralized server setting
8 model_css = torch.nn.Linear(2,1)
9
10 # Choose SGD Optimization for the model
11 opt = torch.optim.SGD(params = model_css.parameters(), lr = 0.01)
```

Design train function to train the model

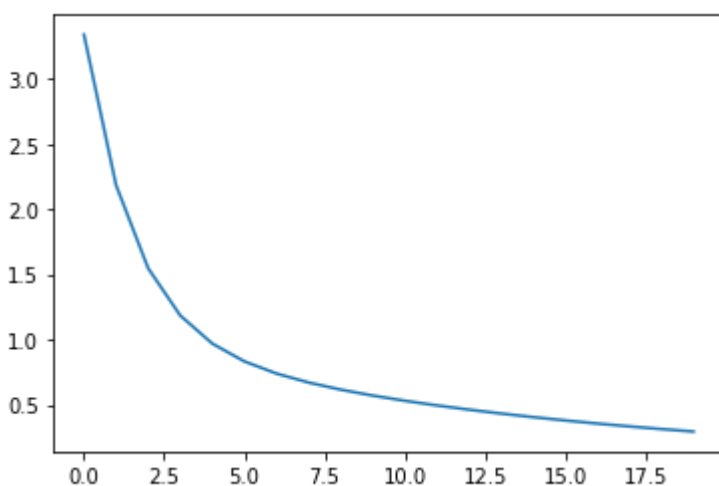
In [19]:

```
1 # Design train function to train the model
2 def train(epochs = 20):
3     # Store each epoch's loss as a list for print
4     loss_data = list()
5     # Each training epoch
6     for iter in range(epochs):
7         opt.zero_grad()
8         pred = model_css(data)
9         # calculate loss
10        loss = ((pred - target) ** 2).sum()
11        # backward
12        loss.backward()
13        opt.step()
14        # Store each epoch's loss in the list
15        loss_data.append(loss.data)
16    return loss_data
```

Now, let's plot each epoch's loss of training model

In [20]:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 loss_data = list()
5 x = list()
6 y = list()
7 loss_data = train()
8
9 for i in range(0, len(loss_data)):
10    x.append(i)
11    y.append(loss_data[i].numpy())
12
13 plt.figure()
14 plt.plot(x,y)
15 plt.show()
16 print("The final model's loss is: ", loss_data[9])
```



The final model's loss is: tensor(0.5696)

Now, let's train the same model with the same dataset in a federated learning setting. Let's create some virtual clients firstly.

In [21]:

```
1 import torch
2 import syft as sy
3 import copy
4 import numpy
5 hook = sy.TorchHook(torch)
6
7 # Create three virtual workers
8 # The first one is "Bob"
9 bob = sy.VirtualWorker(hook, id = "bob")
10 # The second one is "Alice"
11 alice = sy.VirtualWorker(hook, id = "alice")
12 # The third one is secure worker for aggregation purpose
13 secure_worker = sy.VirtualWorker(hook, id = "secure_worker")
```

WARNING:root:Torch was already hooked... skipping hooking process

Now we let's create the mini dataset and partition this data to two virtual clients. Each client will holds partial dataset to simulate federated learning setting that each client use their own local data to train the model.

In [22]:

```
1 # Create a mini dataset
2 data = torch.tensor([[0,0],[0,1],[1,0],[1,1.], [0.1,0.1],[0.1,1.1],[1.1,0.1],[1.1,1.1]])
3 target = torch.tensor([[0],[0],[1],[1.], [0],[0],[1],[1.]], requires_grad = True)
4 # Partition the data to Bob and Alice
5 # Partition the first four data to Bob
6 bobs_data = data[0:4].send(bob)
7 # Partition the first four labels to Bob
8 bobs_target = target[0:4].send(bob)
9 # Partition the last four data to Alice
10 alices_data = data[4:].send(alice)
11 # Partition the last four labels to Alice
12 alices_target = target[4:].send(alice)
```

Build the federated learning global model.

In [23]:

```
1 # Create a model for federated learning setting
2 model_fls = torch.nn.Linear(2,1)
```

Now, allocate the global model to virtual clients, and set model optimizer for virtual clients.

In [24]:

```
1 # Transfer
2 bobs_model = model_fls.copy().send(bob)
3 alices_model = model_fls.copy().send(alice)
4
5 # Set optimizer for every worker
6 # Set optimizer for Bob
7 bobs_opt = torch.optim.SGD(params = bobs_model.parameters(), lr = 0.1)
8 # Set optimizer for Alice
9 alices_opt = torch.optim.SGD(params = alices_model.parameters(), lr = 0.1)
```

Bob and Alice train their models separately

In [25]:

```
1  # Set training epochs
2  epochs = 10
3  bobs_loss_list = list()
4  alices_loss_list = list()
5  for i in range(epochs):
6
7      # Start training Bob's model
8      bobs_opt.zero_grad()
9      bobs_pred = bobs_model(bobs_data)
10     bobs_loss = ((bobs_pred - bobs_target) ** 2).sum()
11     bobs_loss.backward()
12     bobs_opt.step()
13     bobs_loss = bobs_loss.get().data
14
15     # Start training Alice's model
16     alices_opt.zero_grad()
17     alices_pred = alices_model(alices_data)
18     alices_loss = ((alices_pred - alices_target) ** 2).sum()
19     alices_loss.backward()
20     alices_opt.step()
21     alices_loss = alices_loss.get().data
22     # Show the loss of Bob's model and Alice's model
23     bobs_loss_list.append(bobs_loss.numpy())
24     alices_loss_list.append(alices_loss.numpy())
```

When Bob and Alice finish training their models, they upload their models to the secure worker for aggregation

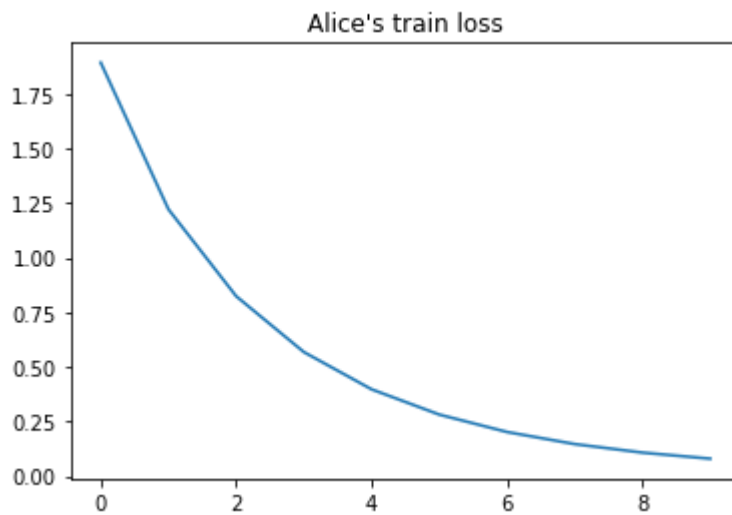
In [26]:

```
1  # Move Both Alice's model and Bob's model to the secure worker for aggregation
2  alices_model.move(secure_worker)
3  bobs_model.move(secure_worker)
4  with torch.no_grad():
5      model_fls.weight.set_(((alices_model.weight.data + bobs_model.weight.data) /
6      model_fls.bias.set_(((alices_model.bias.data + bobs_model.bias.data)).get()))
```

Now, show parameters and model performance of Bob, Alice and global models.

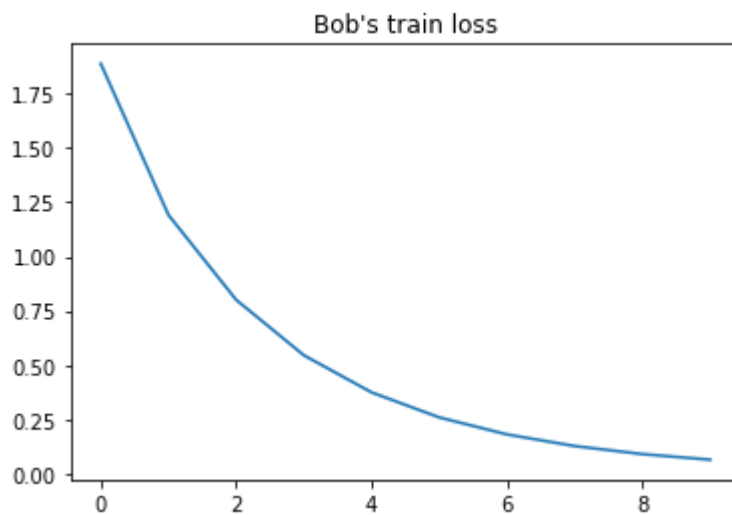
In [27]:

```
1 plt.title("Alice's train loss")
2 plt.plot(np.arange(len(alices_loss_list)), alices_loss_list)
3 plt.show()
```



In [28]:

```
1 plt.title("Bob's train loss")
2 plt.plot(np.arange(len(bobs_loss_list)), bobs_loss_list)
3 plt.show()
```



Now, let's print parameters of Alice's, Bob's and the global models.

In [29]:

```
1 print('\n alices_model:')
2 print((alices_model.weight.data + 0).get())
3 print((alices_model.bias.data + 0).get())
4
5 print('\n boss_model:')
6 print((bobs_model.weight.data + 0).get())
7 print((bobs_model.bias.data + 0).get())
8
9 print('\nmodel:')
10 print(model_fls.weight.data)
11 print(model_fls.bias.data)
12
13 preds = model_fls(data)
14 loss = ((preds - target) ** 2).sum()
15
16 print('\nmean result:')
17 print(preds)
18 print(target)
19 print(loss.data)
```

```
alices_model:
tensor([[ 0.7692, -0.0574]])
tensor([0.1001])
```

```
boss_model:
tensor([[ 0.7928, -0.0337]])
tensor([0.1429])
```

```
model:
tensor([[ 0.7810, -0.0456]])
tensor([0.2430])
```

```
mean result:
tensor([[0.2430],
        [0.1974],
        [1.0240],
        [0.9784],
        [0.3165],
        [0.2710],
        [1.0975],
        [1.0520]], grad_fn=<AddmmBackward>)
tensor([[0.],
        [0.],
        [1.],
        [1.],
        [0.],
        [0.],
        [1.],
        [1.]], requires_grad=True)
tensor(0.2849)
```

4. Federated learning with DPSGD

In this task, we will use DPSGD as our optimizer in federated learning.

In [30]:

```
1 import torch
2 import syft as sy
3 import copy
4 import numpy
5 import DPSGD
6 import matplotlib.pyplot as plt
7
8 hook = sy.TorchHook(torch)
9
10 # Create three virtual workers
11 # The first one is "Bob"
12 bob = sy.VirtualWorker(hook, id = "bob")
13 # The second one is "Alice"
14 alice = sy.VirtualWorker(hook, id = "alice")
15 # The third one is secure worker for aggregation purpose
16 secure_worker = sy.VirtualWorker(hook, id = "secure_worker")
```

WARNING:root:Torch was already hooked... skipping hooking process

In [31]:

```
1 # Create a mini dataset
2 data = torch.tensor([[0,0],[0,1],[1,0],[1,1], [0.1,0.1],[0.1,1.1],[1.1,0.1],[1.1,1.1]])
3 target = torch.tensor([[0],[0],[1],[1], [0],[0],[1],[1]], requires_grad = True)
4 # Partition the data to Bob and Alice
5 # Partition the first four data to Bob
6 bobs_data = data[0:4].send(bob)
7 # Partition the first four labels to Bob
8 bobs_target = target[0:4].send(bob)
9 # Partition the last four data to Alice
10 alices_data = data[4:].send(alice)
11 # Partition the last four labels to Alice
12 alices_target = target[4:].send(alice)
```

In [32]:

```
1 # Create a model for federated learning setting
2 model_fls_dpsgd = torch.nn.Linear(2,1)
```

In [33]:

```
1# Transfer
2bobs_model = model_fls_dpsgd.copy().send(bob)
3alices_model = model_fls_dpsgd.copy().send(alice)
4
5# Set optimizer for every worker
6# Set optimizer for Bob
7bobs_opt = DPSGD.DPSGD(params = bobs_model.parameters(),lr = 0.1,batch_size = 1, (
8# Set optimizer for Alice
9alices_opt = DPSGD.DPSGD(params = alices_model.parameters(),lr = 0.1,batch_size =
10
```

In [34]:

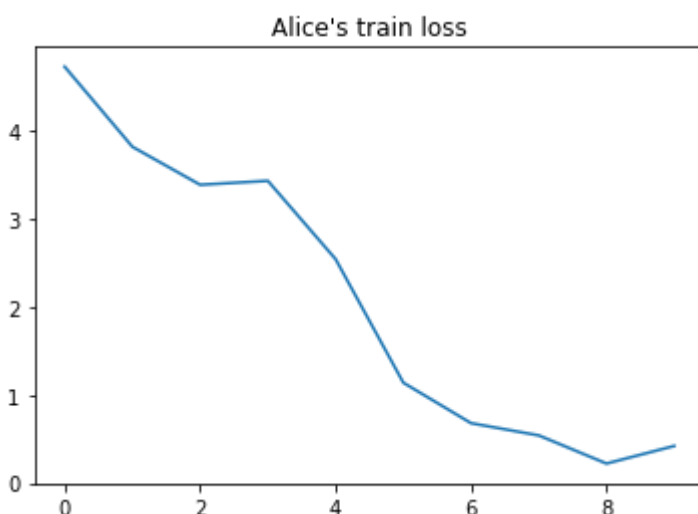
```
1 # Set training epochs
2 epochs = 10
3 bobs_loss_list = list()
4 alices_loss_list = list()
5 for i in range(epochs):
6
7     # Start training Bob's model
8     bobs_opt.zero_grad()
9     bobs_pred = bobs_model(bobs_data)
10    bobs_loss = ((bobs_pred - bobs_target) ** 2).sum()
11    bobs_loss.backward()
12    bobs_opt.step()
13    bobs_loss = bobs_loss.get().data
14
15    # Start training Alice's model
16    alices_opt.zero_grad()
17    alices_pred = alices_model(alices_data)
18    alices_loss = ((alices_pred - alices_target) ** 2).sum()
19    alices_loss.backward()
20    alices_opt.step()
21    alices_loss = alices_loss.get().data
22    # Show the loss of Bob's model and Alice's model
23    bobs_loss_list.append(bobs_loss.numpy())
24    alices_loss_list.append(alices_loss.numpy())
```

In [35]:

```
1 # Move Both Alice's model and Bob's model to the secure worker for aggregation
2 alices_model.move(secure_worker)
3 bobs_model.move(secure_worker)
4 with torch.no_grad():
5     model_fls.weight.set_(((alices_model.weight.data + bobs_model.weight.data) /
6     model_fls.bias.set_(((alices_model.bias.data + bobs_model.bias.data)).get())
```

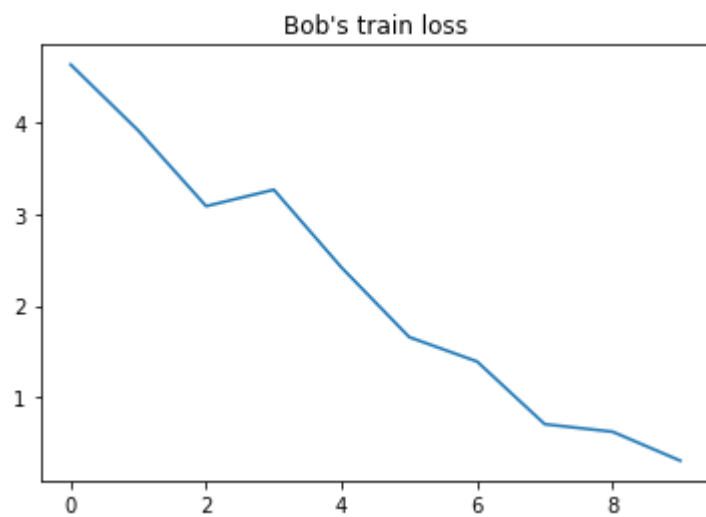
In [36]:

```
1 plt.title("Alice's train loss")
2 plt.plot(numpy.arange(len(alices_loss_list)), alices_loss_list)
3 plt.show()
```



In [37]:

```
1 plt.title("Bob's train loss")
2 plt.plot(numpy.arange(len(bobs_loss_list)), bobs_loss_list)
3 plt.show()
```



In [38]:

```
1 print('\n alices_model:')
2 print((alices_model.weight.data + 0).get())
3 print((alices_model.bias.data + 0).get())
4
5 print('\n boss_model:')
6 print((bobs_model.weight.data + 0).get())
7 print((bobs_model.bias.data + 0).get())
8
9 print('\nmodel:')
10 print(model_fls.weight.data)
11 print(model_fls.bias.data)
12
13 preds = model_fls(data)
14 loss = ((preds - target) ** 2).sum()
15
16 print('\nmean result:')
17 print(preds)
18 print(target)
19 print(loss.data)
```

```
alices_model:
tensor([[ 0.7683, -0.4489]])
tensor([0.1892])
```

```
boss_model:
tensor([[ 0.8185, -0.0732]])
tensor([-0.0881])
```

```
model:
tensor([[ 0.7934, -0.2610]])
tensor([0.1011])
```

```
mean result:
tensor([[ 0.1011],
        [-0.1599],
        [ 0.8945],
        [ 0.6334],
        [ 0.1543],
        [-0.1067],
        [ 0.9477],
        [ 0.6867]], grad_fn=<AddmmBackward>)
tensor([[0.],
        [0.],
        [1.],
        [1.],
        [0.],
        [0.],
        [1.],
        [1.]], requires_grad=True)
tensor(0.3174)
```

5. Conclusion

In this lab, we use an easy demo to implement federated learning. We use PySyft to create virtual workers to collaboratively train a federated learning task.

In this lab, we have learnt:

In this lab, we have learned:

1. We use pip3 to install PySyft;
2. Federated learning involves some workers to collaboratively train a machine learning task with their own data;
3. Use PySyft to create virtual workers participating federated learning;
4. Use DPSGD in federated learning setting.

In []:

1	
---	--